EXP - 08

Design, synthesis, and analysis of sequential circuits.

Objective:

The objective of RTL synthesis of a sequential digital circuit is to transform a high-level functional description of the circuit (Verilog) into a lower-level, technology-specific gate-level representation that can be implemented in hardware. Further, students are able to run the simulation using the TCL script for synthesis of any sequential circuit.

Exercise Problem: 4-bit PIPO (parallel – in parallel -out) shift register circuit.

- Write a verilog code for 4-bit PIPO shift register circuit
- Simulate with nclaunch
- Simulate without nclaunch
- Synthesis using tcl command

Modelling Style: Sequential Modelling

Tools required:

- Functional Simulator
- Incisive Simulator
- Synthesis: Genus
- Physical Design: Innovus

Design Information and Bock Diagram

A 4-bit PIPO (Parallel In Parallel Out) shift register is a digital circuit commonly used in digital systems for various purposes such as data storage, parallel-to-serial or serial-to-parallel conversion, and delay elements.

A 4-bit PIPO shift register consists of four flip-flops (memory cells) capable of storing binary data. Each flip-flop stores one bit of data, allowing the register to hold a 4-bit binary word. The register has four parallel input lines, one for each bit. When new data is presented at the input, it is loaded simultaneously into all four flip-flops, updating the contents of the register. The register has four parallel output lines, one for each bit. These output lines allow the current contents of the register to be read out in parallel, providing access to the stored data.

The PIPO shift register typically doesn't perform shifting operations. Instead, it holds the data statically until new data is loaded into it. The "Parallel-In Parallel-Out" nature of the register means that data is both input and output simultaneously, without any shifting.

Applications: PIPO shift registers find applications in various digital systems where parallel data storage and retrieval are necessary. They can be used in microcontrollers, microprocessors, digital signal processing, and communication systems for tasks such as data buffering, temporary storage, and interfacing between parallel and serial data streams.

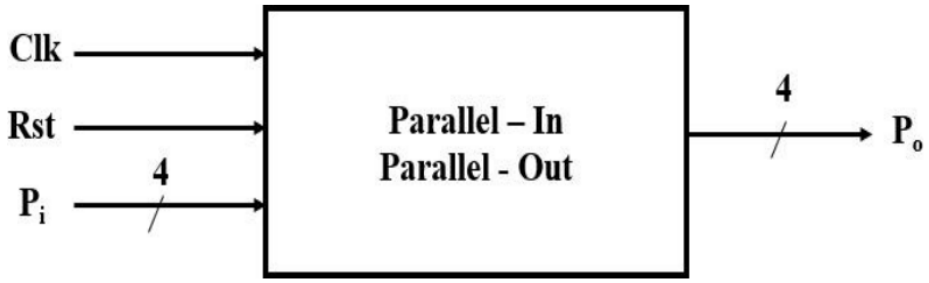


Fig. 1: Block diagram of 4-bit PIPO

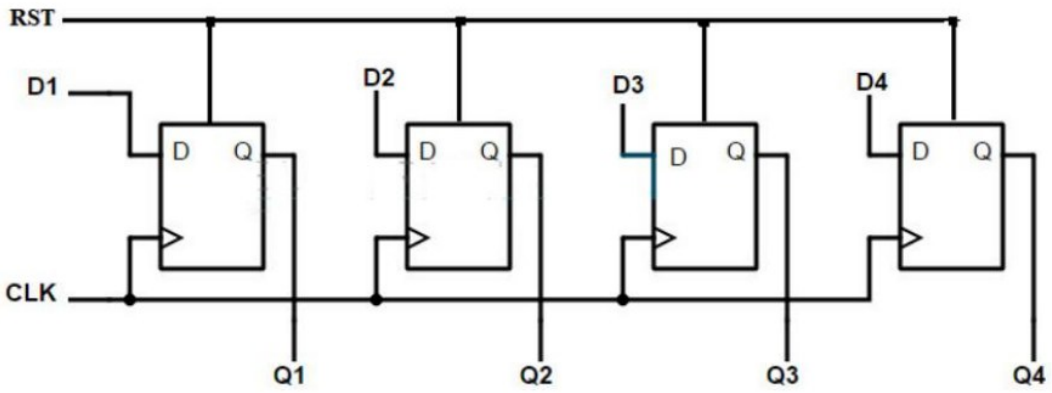


Fig. 2: Block diagram using D flipflop

Tool Required:

- Functional Simulation: Incisive Simulator (ncvlog, ncelab, ncsim).
- Synthesis: Genus

Creating a Workspace:

- In Desktop Create a folder and name it as Digital_Vlsi (give folder name without any space).
- Create a new sub-directory and name it as pipo_seq for the design and open a terminal from the sub-directory.

Functional Simulation:

- Invoke the cadence environment by type the below commands • csh (Invokes C-Shell)
- source /home/install/cshrc (mention the path of the tools) (The path of cshrc could vary depending on the installation destination as /home/install/or /home etc.)

Creating Source Codes:

- In the Terminal, type gedit .v
- A blank document opens up into which the following source code can be typed down.

Source Code

```
module pipo_seq(clk,rst, pi, po);
input clk,rst;
input [3:0] pi;
output [3:0] po;
wire [3:0] pi;
reg [3:0] po;

always @(posedge clk)

begin
if (rst)
po<= 4'b0000;
else
po <= pi;
end

endmodule
```

Creating Test Bench: Similarly, create your test bench using gedit <filename_tb>.v to open a new blank document.

```
module pipo_tb;
reg clk;
reg rst;
reg [3:0] pi;
wire [3:0] po;

pipo_seq uut (.clk(clk),.rst(rst),.pi(pi),.po(po) );

initial begin
clk = 0;
rst = 0;
pi = 4'b0001;
#5 rst=1'b1;
#5 rst=1'b0;
#10 pi=4'b1001;
#10 pi=4'b1010;
#10 pi=4'b1011;
#10 pi=4'b1110;
#10 pi=4'b1111;
#10 pi=4'b0000;
end

always #5 clk = ~clk;

initial #100 $finish;
initial
$monitor ("Time: %0t, pi: %b, po: %b", $time, pi, po);

endmodule
```

To Launch Simulation tool

- linux:/> nclaunch -new&
  // "-new" option is used for invoking NCVERILOG for the first time for any design
- linux:/> nclaunch&
  // On subsequent calls to NCVERILOG
- It will invoke the nclaunch window for functional simulation we can compile, elaborate and simulate it using Multiple.

To perform the function simulation, the following three steps are involved Compilation, Elaboration and Simulation.

Step 1: Compilation: Process to check the correct Verilog language syntax and usage

Inputs: Supplied are Verilog design and test bench codes

Outputs: Compiled database created in mapped library if successful, generates report else error reported in log file

Steps for compilation:

1. Create work/library directory (most of the latest simulation tools creates automatically).
2. Map the work to library created (most of the latest simulation tools creates automatically).
3. Run the compile command with compile options.

- Left side select the file and in Tools : launch verilog compiler with current selection will get enable. Click it to compile the code
- Worklib is the directory where all the compiled codes are stored while Snapshot will have output of elaboration which in turn goes for simulation

- After compilation it will come under worklib you can see in right side window.
- Select the test bench and compile it. It will come under worklib. Under Worklib you can see the module and test-bench.
- The cds.lib file is an ASCII text file. It defines which libraries are accessible and where they are located. It contains statements that map logical library names to their physical directory paths. For this Design, you will define a library called "worklib".

Step 2: Elaboration: To check the port connections in hierarchical design

Inputs: Top level design/test bench Verilog codes

Outputs: Elaborate database updated in mapped library if successful, generates report else error reported in log file

Steps for elaboration – Run the elaboration command with elaborate options

1. It builds the module hierarchy.
2. Binds modules to module instances.
3. Computes parameter values.
4. Checks for hierarchical names conflicts.
5. It also establishes net connectivity and prepares all of this for simulation.

- After elaboration the file will come under snapshot. Select the test bench and elaborate it.

Step 3: Simulation: Simulate with the given test vectors over a period of time to observe the output behaviour.

Inputs: Compiled and Elaborated top level module name.

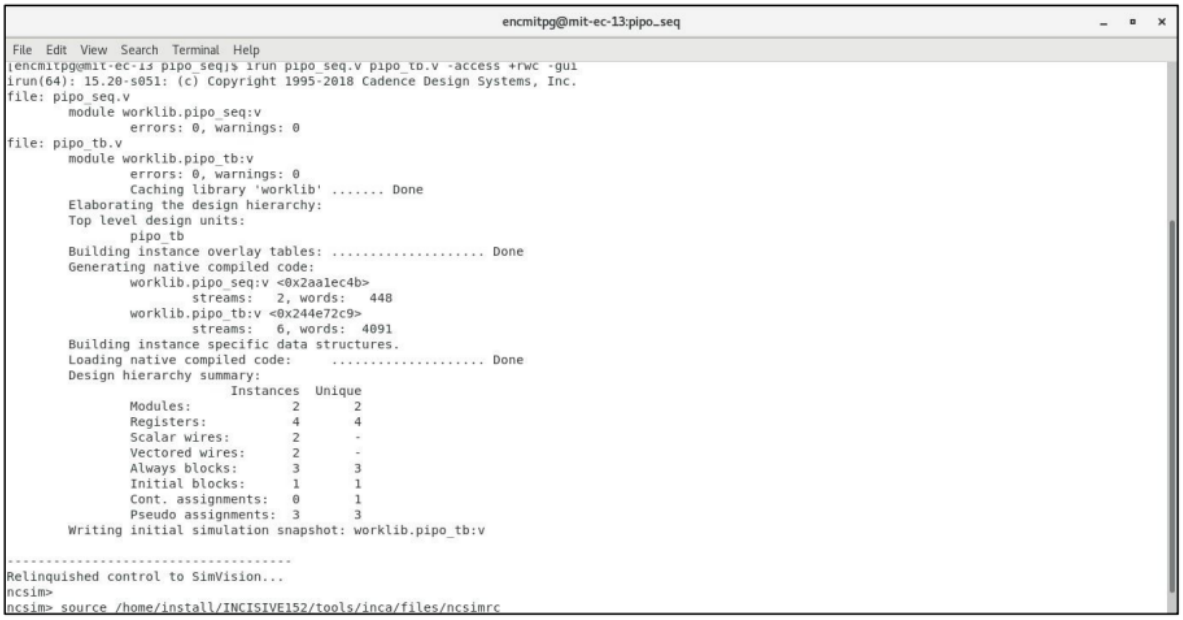
Outputs: Simulation log file, waveforms for debugging.

Simulation allow to dump design and test bench signals into a waveform.

Steps for simulation – Run the simulation command with simulator options.

RUN WITHOUT NCLAUNCH: Instead of nclaunch, design file and testbench can be run using single irun command.

```
[encmitpg@mit-ec-13 pipo_seq]$ irun pipo_seq.v pipo_tb.v -access +rwc -gui
```



Synthesize the design using Constraints and analyse reports, critical path and Max Operating Frequency

Step 1: Getting Started

- Make sure you close out all the Incisive tool windows first.
- Synthesis requires three files as follows,
  1. Liberty Files (.lib)
  2. Verilog/VHDL Files (.v or .vhdl or .vhd)
  3. SDC (Synopsis Design Constraint) File (.sdc)

Step 2 : Creating an SDC File

- In your terminal type “gedit counter_top.sdc” to create an SDC File if you do not have one.
- The SDC File must contain the following commands:

Constraint file:

i. create_clock -name clk -period 2 -waveform {0 1} [get_ports "clk"]
ii. set_clock_transition -rise 0.1 [get_clocks "clk"]
iii. set_clock_transition -fall 0.1 [get_clocks "clk"]
iv. set_clock_uncertainty 0.01 [get_ports "clk"]
v. set_input_delay -max 0.3 [get_ports "pi"] -clock [get_clocks "clk"]
vi. set_output_delay -max 0.3 [get_ports "po"] -clock [get_clocks "clk"]

i→ Creates a Clock named "clk" with Time Period 2ns and On Time from t=0 to t=1.
ii, iii → Sets Clock Rise and Fall time to 100ps.
iv → Sets Clock Uncertainty to 10ps.
v, vi → Sets the maximum limit for I/O port delay to 1ps

Step 3 : Performing Synthesis

- The Liberty files are present in the below path, /home/install/FOUNDRY/digital/<Technology_Node_number>nm/dig/lib/
- The Available technology nodes are 180nm ,90nm and 45nm.
- In the terminal, initialise the tools with the following commands if a new terminal is being used.
  - csh
  - source /home/install/cshrc
- The tool used for Synthesis is "Genus". Hence, type "genus -gui" to open the tool.
- The Following are commands to proceed :

1. read_libs /home/install/FOUNDRY/digital/90nm/dig/lib/slow.lib
2. read_hdl seq.v
3. elaborate
4. read_sdc pipo.sdc //reading top level sdc
5. set_db syn_generic_effort medium // effort level to medium for generic, mapping
6. set_db syn_map_effort medium
7. set_db syn_opt_effort medium
8. syn_generic
9. syn_map
10. syn_opt //Performing Synthesis Mapping and Optimisation
11. report_timing > seq_timing.rep
    //Generates Timing report for worst datapath and dumps into file
12. report_area > seq_area.rep
    //Generates Synthesis Area report and dumps into a file
13. report_power > seq_pwr.rep
    //Generates Power Report [Pre-Layout]
14. write_hdl>seq_net.v //Creates readable Netlist File
15. write_sdc > seq.sdc //Creates Block Level SDC
16. gui_show

Genus Script file with .tcl file Extension

Make sure that the following file need to be present in the directory where the simulation is performed.

```
[150205105@aust synthesis_lab]$ tree
.
|-- EDI_files
|   |-- lef
|   |   `-- gsclib045.lef
|   |-- libs
|   |   |-- fast.lib
|   |   |-- slow.lib
|   |   `-- typical.lib
|   `-- others
|       `-- capTable
|-- input_files
|   `-- alu_4bit.v
`-- synthesis_cmd.tcl

5 directories, 7 files
```

Make sure the following commands are present inside the tcl (name.tcl) file

	Commands	Description
1	set_db init_lib_search_path EDI_files/libs/	Sets the value of a specific attribute. Here we are setting directory name where all the timing libraries are located.
2	set_db library slow.lib	Sets which timing library will be used while mapping
3	set_db lef_library EDI_files/lef/gsclib045.lef	Sets lef file of a target technology
4	set_db hdl_search_path input_files	Sets the directory name where RTL is located
5	read_hdl alu_4bit.v	Loads the design with pre-synthesized RTL
6	elaborate	Creates a design from Verilog module. Undefined modules are labeled as unresolved and treated as blackbox
7	set_top_module alu_4bit	Sets top module name
8	current_design alu_4bit	Changes the current directory in the design hierarchy to the specified design

9	write_hdl > alu_4bit_elaborated.v	Creates a structural netlist using generic/mapped logic
10	create_clock -name clk -period 10 [get_ports clk]	Creates a clock named "clk" having 10ns period in a specific port "clk"
11	set_clock_uncertainty -setup 0.5 [get_clocks clk]	Sets uncertainty value for the clocks while calculating setup
12	set_clock_uncertainty -hold 0.5 [get_clocks clk]	Sets uncertainty value for the clocks while calculating hold
13	set_max_transition 2 [get_ports clk]	Sets maximum allowable transition time for changing logic state to 2ns for data path
14	set_clock_transition -min -fall 0.5 [get_clocks clk]	Sets minimum allowable clock transition time to 0.5ns for switching logic state from high to low for clock path
15	set_clock_transition -min -rise 0.5 [get_clocks clk]	Sets minimum allowable clock transition time to 0.5ns for switching logic state from low to high for clock path
16	set_clock_transition -max -fall 0.5 [get_clocks clk]	Sets maximum allowable clock transition time to 0.5ns for switching logic state from high to low for clock path
17	set_clock_transition -max -rise 0.5 [get_clocks clk]	Sets maximum allowable clock transition time to 0.5ns for switching logic state from low to high for clock path
18	set_clock_groups -name original -group [list [get_clocks clk]]	Defines groups of specific clocks
19	set DRIVING_CELL BUFX8	Defines driving cell name which will drive the input ports of the design
20	set DRIVE_PIN {Y}	Defines driver pin of the driving cell
21	set_driving_cell -lib_cell $DRIVING_CELL -pin $DRIVE_PIN [all_inputs]	Sets driving cell properties for all the input ports
22	set_max_fanout 10 [current_design]	Sets maximum allowable fanout number to 10
23	set_load 0.5 [all_outputs]	Sets load capacitance of the output ports of the design
24	set_operating_conditions slow	Sets operating condition for delay calculation
25	set_input_delay -max 0.5 [all_inputs]	Synthesis tool assumes the data is launched by a positive edge triggered flop from the external logic

		(and the maximum input delay for the setup analysis is 0.5ns)
26	set_output_delay -max 0.5 [all_outputs]	Synthesis tool assumes the data is captured by a positive edge triggered flop in the external logic (and the maximum output delay for the setup analysis is 0.5ns)
27	remove_assign -buffer_or_inverter BUFX16 -design [current_design]	Removes assign statement using BUFX16 cell
28	syn_generic	Performs generic synthesis
29	write_hdl > alu_4bit_generic.v	Creates a structural netlist using generic logic after generic synthesis
30	synthesize -to_mapped	Performs mapping using target timing library
31	write_hdl > alu_4bit_post_synthesis.v	Creates a structural netlist using mapped logic after mapping
32	remove_assigns_without_opt -buffer_or_inverter BUFX12 -verbose	Removes assign statement using BUFX12 cell
33	set_remove_assign_options -buffer_or_inverter BUFX12 -verbose	Sets buffer or inverter cell to remove assign statements
34	write -mapped > alu_4bit_mapped.v	Writes mapped netlist for post-synthesis flow
35	write_sdc > alu_4bit.sdc	Writes constraints file for post-synthesis flow

```
# Setting library and RTL paths
set_db / .init_lib_search_path {lib}
set_db / .init_hdl_search_path {rtl}

# Read Lib, RTL and SDC files
set_db / .library "slow.lib"
set DESIGN updowncounter
read_hdl "4bup_down_count.v"
elaborate $DESIGN
check_design -unresolved
read_sdc constraints_top.sdc

# Setting effort medium
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
syn_opt

write_hdl > upDowncounter_netlist.v
write_sdc > upDowncounter_sdc.sdc
# PPA Reports
report_power > upDowncounter_power.rpt
report_gates > upDowncounter_gatecount.rpt
report_timing > upDowncounter_timing.rpt
report_area > upDowncounter_area.rpt
report_qor -levels_of_logic -power -exclude_constant_nets > upDowncounter_qor.rpt
gui_show
```

Steps to create tcl file

Fig. 53: Create tcl file

- Copy the commands from genus folder created in the pipo_seq folder

Fig. 54: TCL file

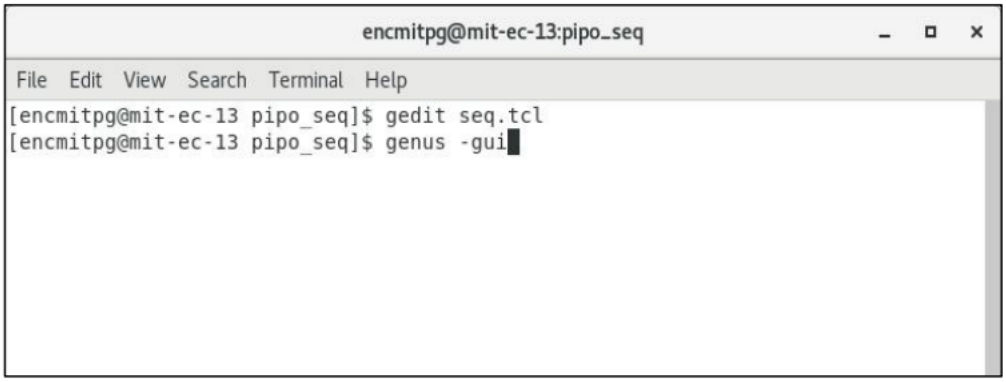


Fig. 55: Open genus

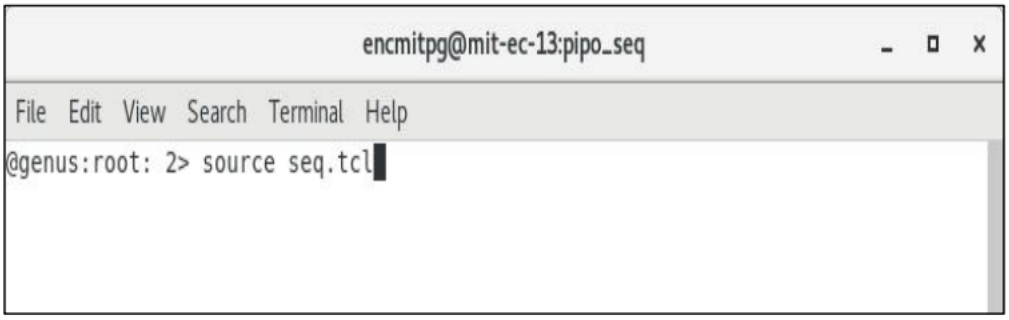


Fig. 56: Source tcl file

Fig. 57: End genus by giving exit command

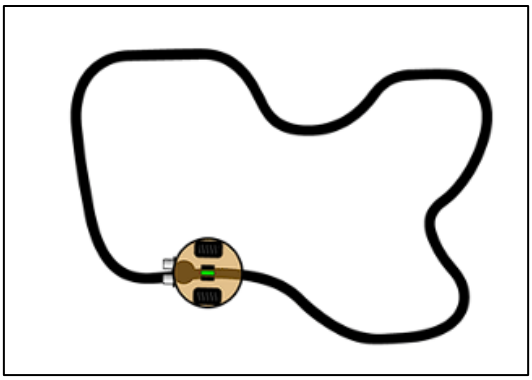
- To show the synthesized output execute the command gui_show
- The GUI window of the genus synthesis output will be opened. If you click and zoom into each block of the circuit, you will be able to view the gate interconnections inside the block.
- **Post Lab Task**
  - Is the testbench module synthesizable?
  - Why is the operating condition of synthesis slow?
  - What is Standard Design Constraints (SDC)?
  - What do LEF and LIB files contain?
  - List the functions of buffer cells in synthesis.
  - Check the function of commands report_power, report_gates, report_timing in Genus

Exercise problem:

1. Do the following points for 4 bit universal shift register
    a. Understand the use and application of 4 bit universal shift register.
    b. Write sequential Verilog code for 4-bit universal shift register and verify the design by simulation.
    c. Do the synthesis and calculate the power area and performance without using the tcl command
    d. Repeat the above problem with TCL command line.
2. Write the Verilog code for Master Slave JK flipflop and do same as exercise problem 1.
3. Write the Verilog code for synchronous mod 8 counter and do same as exercise problem 1.

(Home work)

A Line follower Robot uses 2 Infra-Red (IR) sensors to detect the color of the surface. Sensor output is High when white surface is detected and Low on a black surface. Try to model the Robot controller and then implement its hardware? A Line follower Robot uses 2 Infra-Red (IR) sensors to detect the color of the surface. Sensor output is High when white surface is detected and Low on a black surface. Try to model the Robot controller and then implement its hardware?



Line Follower